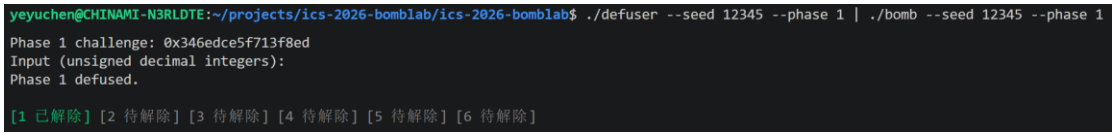
实验报告

运行结果截图：

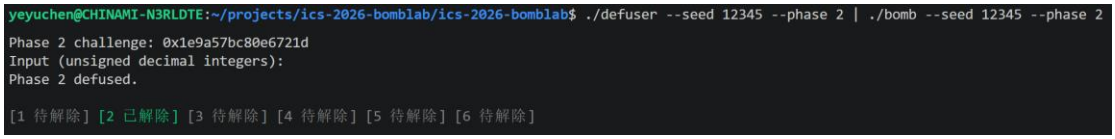
```
yeyuchen@CHINAMI-N3RLDTE:~/projects/ics-2026-bomblab/ics-2026-bomblab$ python3 public_test.py
Phase 1 seed 0: PASS
Phase 1 seed 1: PASS
Phase 1 seed 7: PASS
Phase 1 seed 42: PASS
Phase 1 seed 99: PASS
Phase 1 seed 123: PASS
Phase 1 seed 2026: PASS
Phase 1 seed 65535: PASS
Phase 1 seed 987654321: PASS
Phase 1 seed 18446744073709551615: PASS
Phase 2 seed 0: PASS
Phase 2 seed 1: PASS
Phase 2 seed 7: PASS
Phase 2 seed 42: PASS
Phase 2 seed 99: PASS
Phase 2 seed 123: PASS
Phase 2 seed 2026: PASS
Phase 2 seed 65535: PASS
Phase 2 seed 987654321: PASS
Phase 2 seed 18446744073709551615: PASS
Phase 3 seed 0: PASS
Phase 3 seed 1: PASS
Phase 3 seed 7: PASS
Phase 3 seed 42: PASS
Phase 3 seed 99: PASS
Phase 3 seed 123: PASS
Phase 3 seed 2026: PASS
Phase 3 seed 65535: PASS
Phase 3 seed 987654321: PASS
Phase 3 seed 18446744073709551615: PASS
Phase 4 seed 0: PASS
Phase 4 seed 1: PASS
Phase 4 seed 7: PASS
Phase 4 seed 42: PASS
Phase 4 seed 99: PASS
Phase 4 seed 123: PASS
Phase 4 seed 2026: PASS
Phase 4 seed 65535: PASS
Phase 4 seed 987654321: PASS
Phase 4 seed 18446744073709551615: PASS
Phase 5 seed 0: PASS
Phase 5 seed 1: PASS
Phase 5 seed 7: PASS
Phase 5 seed 42: PASS
Phase 5 seed 99: PASS
Phase 5 seed 123: PASS
Phase 5 seed 2026: PASS
Phase 5 seed 65535: PASS
Phase 5 seed 987654321: PASS
Phase 5 seed 18446744073709551615: PASS
Phase 6 seed 0: PASS
Phase 6 seed 1: PASS
Phase 6 seed 7: PASS
Phase 6 seed 42: PASS
Phase 6 seed 99: PASS
Phase 6 seed 123: PASS
Phase 6 seed 2026: PASS
Phase 6 seed 65535: PASS
Phase 6 seed 987654321: PASS
Phase 6 seed 18446744073709551615: PASS
Public tests: 60/60
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
```

额外 seed 验证截图：

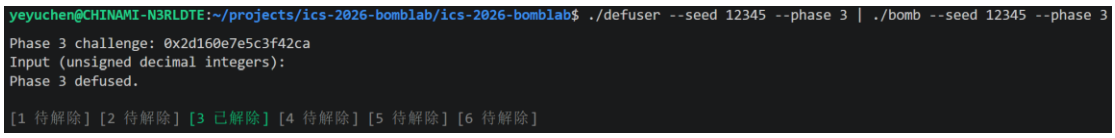
Phase1:



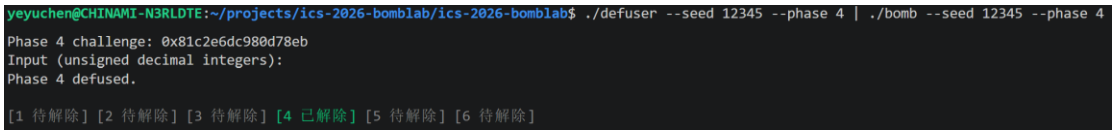
Phase2:



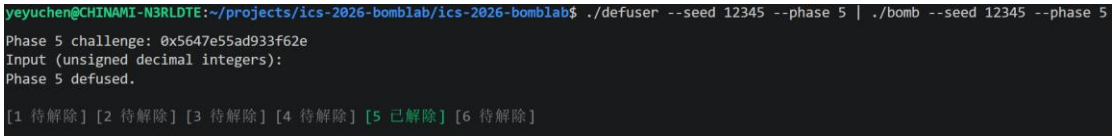
Phase3:



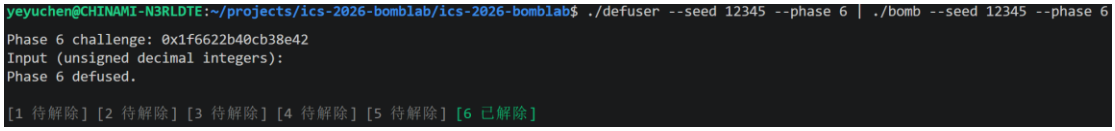
Phase4:



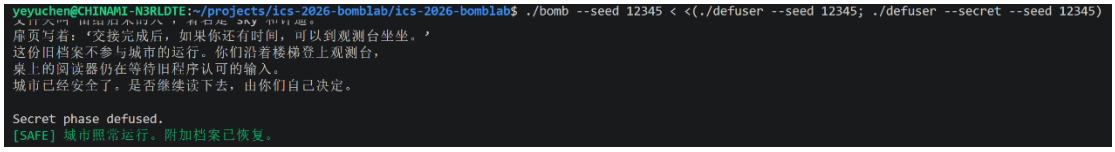
Phase5:



Phase6:



Secret:



Phase1 推理过程

一、确定结构。 phase_01 的失败出口在 4020b3，先复位栈再执行 xor %eax,%eax；4020b1 与 4020d6 两条 jne 都指向它。成功路径在 4020e3 用 sete 之后顺序落到 4020e6 的收尾。函数本体不足二十条指令，只做格式校验、解析、一次调用、一次比较，所有算术集中在 4020db 的 call 目标 402050，即 p1_compute。

二、确定输入个数。 40209d 的 mov $0x1,%esi 给出字段数一；4020c6 的 mov $0x403189,%esi 是格式串，用 gdb -batch -ex 'x/s 0x403189' ./bomb 打出 "%u"，这个地址

与第四、六关相同；4020d3 的 cmp $1,%eax 与返回值比较。说明只接受一个无符号十进制数，缓冲区是 4020c2 的 lea -0x14(%rbp),%rdx，4020e0 的 cmp 与 4020e3 的 sete 构成唯一判定。

三、拆出三段位。 p1_compute 只收到一个 64 位参数，402055 的 mov %rdi,%rax 与 40205e 的 shr $0x20,%rax 取出高 32 位；402058 的 mov %rdi,%rdx 与 402062 的 shr $0x10,%rdx 取出第 16 位以上的部分；40205b 的 movl %edi,%r8d 直接取低 32 位，用的是 movl 而不是移位。说明三段分别是 c0 = challenge 低 32 位、c1 = challenge >> 32、c16 = (challenge >> 16) 的低 32 位，三者互相重叠，c16 横跨 c0 的上半部与 c1 的下半部。

四、三个 step 的串联顺序。 402069 的 movl %eax,%esi 先把 c1 放进第二参，40206e 调用 p1_step0 时第一参 %edi 仍是 challenge 的低 32 位，所以第一步算 step0(c0, c1)；402073 的 movq %rdx,%rsi 把 c16 送进第二参、402076 的 movl %r9d,%edi 把 c1 送进第一参，第二步算 step1(c1, c16)；402080 的 movl %r8d,%esi 把 c0 送进第二参、402083 的 movl %eax,%edi 把上一步的返回值送进第一参，第三步算 step2(step1, c0)。说明三步是串联，前一步的输出就是后一步的第一参数，而 c0 在第一步与第三步各参与一次。

五、中间结果的保活。 402079 的 movl %eax,%ecx 把第一步的结果存进 ecx，此后两次 call 之间没有重新装载；402073 的 movq %rdx,%rsi 依赖 rdx 在第一步之后仍然有效。说明这两处是编译器利用三个 step 都是叶子函数、不碰 ecx 与 rdx 而省下的栈访问，读的时候不能因为中间隔了两次 call 就以为值已失效；要确认可在 40208b 处 p/x $ecx 与 p/x $eax 对照。

六、三个 step 的公式。 p1_step0 的 402024 是 leal (%rdi,%rdi,4)，即五倍，402027 的 addl %esi 加上 c1，402029 的 xorl $0x3848 再整体异或；p1_step1 的 402034 是 leal 0x30d0(%rsi)，即 c16 加常数，40203a 的 xorl %edi 异或 c1；p1_step2 的 402044 又是五倍，402047 的 addl $0xef89,%esi 把常数加到 c0 上，40204d 的 xorl 再异或。说明三式为 step0 = (5c0 + c1) ^ 0x3848、step1 = (c16 + 0x30d0) ^ c1、step2 = 5step1 ^ (c0 + 0xef89)。

七、常数的位置。 step0 的 0x3848 是在加法之后整体异或，step2 的 0xef89 是先加到 c0 上再参与异或，前者异或的是一整项，后者只改异或的右操作数。

八、最外层合并。 40208b 的 xorl %ecx,%eax 把第一步的结果异或到最后一步的返回值上。说明总式为 result = step2 ^ step0。

Phase2 推理过程

一、确定结构。 打开 phase_02.S 后我把所有 jne 的目标统计了一遍：三次跳 402115，一次跳 402128，一次跳 402178。402115 后面紧跟 xor %ebx,%ebx 然后直接收尾，说明它是唯一的失败出口；另外两处都是往前跳。于是函数被切成“前置校验 → 解析 → 检查① → 循环”四段。

二、确定输入个数。 gdb -batch -ex 'x/s 0x40317a' ./bomb 打出 "%u %u %u %u %u %u"，是 sscanf 的第二个参数；同时 mov $0x6,%esi 出现在 call valid_u32_fields 之前。说明输入为

6 个无符号十进制数。

三、输入存放结构。 静态能看到 sscanf 的六个输出缓冲区地址相差 4 字节，以及循环里 -0x40(%rbp,%rdx,4) 是比例寻址。为了确认顺序，输入一组互不相同的值 11 22 33 44 55 66，停在 call sscanf 之前，先 x/s $rsi 确认格式串、set $v0 = $rdx，再 ni 过掉调用，然后 p/d $eax 得 6、x/6uw $v0 得 11 22 33 44 55 66。说明值按输入顺序，存放的结构是一个数组 uint32_t v[6] ，且 rbp-0x40 是 v[0]。

四、第一项。 该段无分支，可直接确定操作；随后 cmp %eax,%esi 将结果与第一项比较。参与比较的 %esi 由前面的 mov -0x40(%rbp),%esi 载入，即 v[0]。据此得到 v[0] = ((challenge >> 6) & 0x1f) + 9

五、循环骨架。 静态读控制指令：402173 的 mov $0x1,%edx 给出下标初值 1，40216e 的 mov $0x9,%ecx 给出移位量初值 9；循环体末尾 402193 的 add $0x1,%rdx 使下标加一，402197 的 add $0x5,%ecx 使移位量加五，随后 40219a 的 cmp $0x6,%rdx 与 40219e 的 jne 402178 构成回边。说明：%rdx 从 1 递增到 5，共 5 轮；%rcx 依次为 9、14、19、24、29，通式为 9+5*(i-1)。

六、循环体内 %esi 出现两次用途不同：phase_02+0x8e 的 lea (%rsi,%rsi,4),%esi 把它当作乘数，phase_02+0x9b 的 mov -0x40(%rbp,%rdx,4),%esi 又把它覆盖成本轮输入项。说明 %esi 在本轮末尾被置为本轮输入项，下一轮进入循环体时即以该项作为乘数，因此六项构成链式递推。

七、数据类型与截断。 在 phase_02+0x8e 的 lea 处读取 p/x $rsi 得 0x26，单步执行后得 0xbe。高 32 位被清零，仅低 32 位变化，说明乘法落在 32 位域，C 中须用 uint32_t 而非 int。challenge 本身是 64 位，shr %cl,%rax 在 64 位上执行，直到 and $0x1f 之后才降为 32 位，两者不可混用。

Phase3 推理过程

一、确定结构。 打开 phase_03.S 后先统计跳转字段中能跳出去的目标：4022d1 与 4022fa 跳到 4022d3，402307 跳到 4022d5，而 4022d5 是共用收尾。说明失败有两条入口，但都返回 0，成功路径也汇到 4022d5；函数被切成校验、解析、第一项、派生、分派、比较六段。

二、确定输入个数。 4022bd 的 mov $0x2,%esi 紧接在 call valid_u32_fields 之前，给出字段数 2；4022f7 的 cmp $0x2,%eax 与 sscanf 的返回值比较。格式串来自 4022ea 的 mov $0x403186,%esi，用 gdb -batch -ex 'x/s 0x403186' ./bomb 打出 “%u %u”。说明输入是两个无符号十进制数。

三、输入存放位置。 sscanf 的参数按调用约定排开：4022ef 的 mov %rbx,%rdi 是输入串，4022ea 的 %esi 是格式串，4022e6 的 lea -0x18(%rbp),%rdx 是第三个参数即 &v[0]，4022e2 的 lea -0x14(%rbp),%rcx 是第四个参数即 &v[1]。两个地址相距 4 字节，说明元素宽 4 字节，第一项落在 rbp-0x18，第二项落在 rbp-0x14。

四、第一项。 4022fc 的 mov %r12d,%edi 取 challenge 的低 32 位，402301 的 and $0x7,%edi 只保留低 3 位，402304 的 cmp %edi,-0x18(%rbp) 与第一项比较。说明 v[0] = challenge & 7，取值只有 0 到 7。4022ff 的 xor %eax,%eax 先把 eax 清零，这解释了 402307 为何能直接跳到共用收尾而不是跳 4022d3。

五、派生值。402309 与 40230c 把 challenge 复制进 rax 与 rsi，40230f 的 shr $0x3,%rax 在 64 位域右移 3 位，402313 的 shr $0x20,%rsi 取出高 32 位，402317 的 xor %eax,%esi 做 32 位异或并把 rsi 的高 32 位清零。说明传给分派函数的值是 x = ((challenge >> 3) & 0xffffffff) ^ (challenge >> 32)。右移在 64 位域执行、异或在 32 位域执行。

六、参数传递。 402319 的 call p3_dispatch 之前没有重新装载参数：%edi 仍是第四步算出的 challenge & 7，%esi 是第五步的 x。分派函数内部 402254 的 mov %edi,%eax 把开关移出，402256 的 mov %esi,%edi 把 x 挪到第一个参数位。说明八个分支函数收到的 %rdi 是 x。

七、跳转表。 40225d 的 notrack jmp *0x405820(,%rax,8) 是基址加下标乘八的绝对地址跳转，402258 的 cmp $0x7,%eax 与 40225b 的 ja 做越界检查，因 datas 已被 and $0x7 限死而实际不可达。用 x/8gx 0x405820 读表并逐项对照桩上的 jmp 目标，开关 0 到 7 依次进入 p3_case_7、p3_case_3、p3_case_6、p3_case_5、p3_case_1、p3_case_0、p3_case_2、p3_case_4。开关值与分支编号并不一致。

八、八个分支。逐条读出：分支 0 为 5x + 0x3d0e5，分支 1 为 3x + 0x179e，分支 2 为 (2x) ^ 0xf387，分支 3 为 (x ^ 0x7ec3) + (x >> 2)，分支 4 为 3x + 0xd8dd，分支 5 为 5x + 0x9501，分支 6 为 (4x) ^ 0x33cf，分支 7 为 (x ^ 0x96df) + (x >> 1)。

九、比较与收尾。 40231e 的 cmp %eax,-0x14(%rbp) 把分派函数的返回值与第二项比较，402321 的 sete %al 只在相等时把 al 置 1，402324 跳到共用收尾。说明 sete 只写 al，eax 的高 24 位仍是分派函数的返回值，作为 bool 返回时只看 al；若按 int 理解会以为返回值可能是带高位的怪数。

Phase4 推理过程

一、确定结构。 打开 phase_04.S 后先找出口：4023c3 的 xor %eax,%eax 后直接收尾，是唯一的失败出口，4023c1 与 4023e6 两条 jne 都指向它；402410 是成功收尾。另外导出 p4_walk。

二、确定输入个数。 4023ad 的 mov $0x1,%esi 紧接在 call valid_u32_fields 之前，给出字段数 1；4023d6 的 mov $0x403189,%esi 是 sscanf 的格式串，用 gdb -batch -ex 'x/s 0x403189' ./bomb 打出 "%u"；4023e3 的 cmp $1,%eax 与返回值比较。说明输入为一个无符号十进制数。

三、确定四个参数。 4023e8 到 402403 这几条交叉排列，要按写入哪个寄存器归类而不是按地址顺序读。4023e8 的 mov %r12,%rcx 与 4023eb 的 mov %r12,%rax 各留一份

challenge；4023ee 的 mov %r12d,%edi 取低 32 位，再由 4023fb 的 and $0x7f,%edi 截到 0 到 127；4023f1 的 xor %esi,%esi 把第二个参数清零；4023fe 的 mov $0x7f,%edx 把第三个参数置 127；4023f7 的 shr $0x7,%rcx 与 4023f3 的 shr $0x20,%rax 分别右移七位与 32 位，最后由 402403 的 xor %eax,%ecx 合成第四个参数。说明调用形式为 p4_walk(challenge & 0x7f, 0, 0x7f, ((challenge >> 7) & 0xffffffff) ^ (challenge >> 32))，即在下标 0 到 127 的整数中查找 challenge 的低七位。

四、辨认递归。402357 与 402375 的 call 目标都是 402330，即函数自身，说明 p4_walk 是递归函数；两处分居条件分支两侧，说明递归有左右两路，对应二分查找的两个子区间。

五、中点和基例。402334 的 mov %edi,%eax 先把目标值存进 eax；402336 到 40233c 的 mov %edx,%edi、sub %esi,%edi、shr $1,%edi、add %esi,%edi 四步把 %edi 变成中点；40233e 的 cmp %eax,%edi 与 402340 的 je 402390 判断中点是否等于目标。说明 mid = b + (c-b)/2 向下取整，且 %edi 原本保存的目标值被挪到 eax，此后 %edi 都代表中点。402390 的 leal 0x73(%rdi),%eax 与 402393 的 xor %ecx,%eax 给出基例返回值 (mid + 0x73) ^ key，此时 %edi 仍是 mid 且与目标值相等，两处可以互换。

六、两个分支。40234e 的 cmp %edi,%eax 计算的是目标减中点，402350 的 jb 跳向 402370，而 402370 的 leal -1(%rdi),%edx 把上界改成中点减一、%esi 未动，说明目标小于中点时搜索左半区间；不跳转则执行 402352 的 leal 1(%rdi),%esi 把下界改成中点加一、%edx 未动，搜索右半区间。

七、回程的累积。402347 的 leal (%rdi,%rcx),%ebx 在递归之前把本层的中点加 key 存入 ebx；ebx 属于 callee-saved，递归调用不会改写它，402361 与 40237f 的 movq -8(%rbp),%rbx 是恢复而非重算。返回后 40235c 的 leal (%rax,%rax,4),%eax 把子结果乘五，40235f 的 xorl %ebx,%eax 异或本层常数，最后右分支在 402366 执行 xorl $2、左分支在 402384 执行 xorl $1。说明递推为：目标小于中点时返回五倍左子树结果再异或中点加 key 再异或一，否则返回五倍右子树结果再异或中点加 key 再异或二。常数是在回程逐层叠加的，经初期时测试发现若误作下行累积写成五倍（子结果异或常数），结果会完全不同。

Phase5 推理过程

一、确定结构。phase_05 的失败出口在 4024a3，先执行 xor %eax,%eax 再走收尾；4024a5 是共用收尾，返回 %eax 原值。4024a1 与 4024ca 跳向 4024a3，4024df 跳向 4024a5。函数本体只做四件事：格式校验、解析、调用一次 p5_traverse、两次比较，自身不含任何算术，所以必须另外导出 4024d3 的 call 目标 402420。

二、确定输入个数。40248d 的 mov $0x2,%esi 紧接在 call valid_u32_fields 之前，给出字段数二；4024ba 的 mov $0x403186,%esi 是 sscanf 的格式串，这个地址与第三关是同一个，用 gdb -batch -ex 'x/s 0x403186' ./bomb 打出 "%u %u"；4024c7 的 cmp $2,%eax 与返回值比较。说明输入是两个无符号十进制数，第一项落在 rbp-0x1c，第二项落在 rbp-0x18，另有一个输出槽在 rbp-0x14，三处相距都是四字节。

三、确定调用形式。4024cc 的 lea -0x14(%rbp),%rsi 是第二个参数，即输出槽的地址；4024d0

的 mov %r12,%rdi 是第一个参数，即 challenge。说明调用形式为 p5_traverse(challenge, &slot)，函数返回一个 32 位值，同时把某个值写入 slot，两个结果分别对应两项输入。

四、两项比较的顺序。 4024d8 的 mov %eax,%edx 先把返回值暂存，4024da 的 xor %eax,%eax 随后把返回值清零，所以 4024df 失败时可以直接跳共用收尾却仍返回 false；4024dc 的 cmp %edx,-0x1c(%rbp) 要求第一项等于返回值。第二段由 4024e1 的 movl -0x14(%rbp),%eax 取出 slot，4024e4 的 cmp %eax,-0x18(%rbp) 与 4024e7 的 sete 要求第二项等于 slot。说明第一项对应遍历的累计值，第二项对应末节点的常数，且第一项不对时就不会检查第二项。

五、表选择与跳数。 40242d 的 and $7,%eax 只留 challenge 低三位，402434 的 mov 0x405b60(,%rax,8),%rdx 以此为下标从八项指针表里取链表头，表项是八字节绝对地址，说明这是一张八项的指针表而不是地址偏移表。402430 的 shr $3,%rcx 与 40245b 的 and $3,%edi 再取第 3、4 两位，作为要额外推进的节点数。说明起始节点由低 3 位选定，实际访问节点数为 1 加上 (challenge >> 3) & 3，最多四个。

六、初值与低字节异或。40243c 到 402447 的 mov %rdi,%rax、shr $6,%rax、shr $0x20,%rdi、xor %edi,%eax 给出累计初值，即 challenge 右移六位的低 32 位与 challenge 高 32 位相异或；40244e 的 xorb $0x91,%al 只改最低字节。因为 0x91 本身不足八位，这条指令与整体异或 0x91 等价，写成 xor %eax 与 xorb %al 得数相同。

七、单节点公式。每个节点执行 acc = 3 * acc + c 再异或 b，节点结构是偏移 0 存 next 指针、偏移 8 与 0xc 各存一个 32 位常数。

八、循环与输出。 402456 的 movl %esi,(%r8) 把本节点的 b 写入 slot，此后每轮 402476 都覆盖一次，所以最终留下的是末节点的 b；这是第二项输入的来源。循环体从 402465 起：402465 的 movq (%rdx),%rdx 沿 next 前进一个节点，40246b 的 addl $1,%ecx 计数，402479 的 cmpl %ecx,%edi 与 40247b 的 jae 在计数不超过跳数时继续。说明跳数为零时 40245e 的 je 直接返回，指针一次也不移动，此时两项输入只依赖起始节点。

九、节点数据。 0x405b60 处的八项依次指向同一条链表的第零到第七个节点，整条链共十二个节点，因此这张表只是同一串数据的不同起点。用 x/8gx 0x405b60 取表，再沿每项的 next 逐项读偏移 8 与 0xc，得到的常数对依次是 2600 与 42397、2950 与 33530、2330 与 58296、60 与 25581、1611 与 58210、533 与 20617、1052 与 64559、2108 与 14904、847 与 3050、1823 与 42938、1316 与 64417、319 与 52207。

Phase6 推理过程

一、确定结构。 phase_06 只有 4025f3 一个失败出口，先 addq $0x10,%rsp 复位栈，再 xor %eax,%eax；4025f1 与 402616 两条 jne 都指向它。成功路径在 402623 用 sete 之后顺序落到 402626 的收尾。函数本体只做格式校验、解析、调用一次 p6_dispatch、一次比较，自身没有算术，所以必须另外导出 40261b 的 call 目标 402580。

二、确定输入个数。 4025dd 的 mov $0x1,%esi 给出字段数为一；402606 的 mov $0x403189,%esi 是格式串，这个地址与第四关相同，用 gdb -batch -ex 'x/s

0x403189' ./bomb 打出 "%u"；402613 的 cmp $1,%eax 与返回值比较。说明只接受一个无符号十进制数，缓冲区是 402602 的 lea -0x14(%rbp),%rdx。402620 的 cmp %eax,-0x14(%rbp) 与 402623 的 sete 是唯一的判定，说明本关只有一个比较，不存在前几关那种双项校验。

三、确定分派的实参。 402589 的 mov %rdi,%rbx 保存 challenge；40258c 的 shr $6,%rdi 得到右移六位的值；402590 与 402593 的 mov %rbx,%rax 加 shr $0x20,%rax 取出高 32 位；402597 的 xorl %eax,%edi 把两者相异或。说明传给算子的是 x = ((challenge >> 6) & 0xffffffff) ^ (challenge >> 32)，低六位不参与这个值。右移在 64 位域完成、异或在 32 位域完成，两者不能混写。

四、两次查表共用一个基址。 402599 的 movq %rbx,%rax 把 challenge 重新装回 rax，4025a3 的 andl $7,%eax 于是取的是低三位而不是上一句用过的高半部，4025a6 的 call *0x405e00(,%rax,8) 以此为下标取函数指针。4025a0 的 andl $0x38,%ebx 与 4025b4 的 call *0x405e00(%rbx) 则是把第 3 到第 5 位当成字节偏移用，因为 0x38 等于七乘八，这个偏移恰好落在同一张表的另一项上。说明两个算子来自同一张八项表，第一个下标是 challenge & 7、第二个是 (challenge >> 3) & 7，第二次调用无需再乘八。这是本关最容易看漏的一处，容易误以为程序里有两张表。

五、两次调用之间的变换。 4025ad 的 xorl $0xe892,%eax 修改第一次的返回值，4025b2 的 movl %eax,%edi 把它作为第二次调用的实参，4025bf 的 addl $0xcb90,%eax 把常数加在第二次的返回值上。说明整体是 f2(f1(x) ^ 0xe892) + 0xcb90，中间那次异或属于算子之间的串联，不属于任何一个算子；把 0xe892 挪到最外层会得到完全不同的值。

六、算子表的内容。 用 x/8gx 0x405e00 读出的八项依次是 402520、402560、4024f0、402500、402530、402540、4024f0、4024f0。前五项各指向一个不同的 p6_op_ 函数，后面三个下标中有两个重复指向 4024f0。说明这张表不是置换，下标 2、6、7 都落到同一个算子，按下标顺序猜算子必错，必须逐项对照栈上的目标地址。

七、六个算子的公式。逐个读：p6_op_0 是 4024f6 的 roll $7 再 4024f9 的 xor $0x3e28；p6_op_1 是 402506 的 shr $7、402509 的 xor %edi 与 40250b 的 add $0xfc35；p6_op_2 只有 402526 的 rorl $1；p6_op_3 是 402534 的 leal 0x7a4e(%rdi) 加 40253a 的 shr $3 再 40253d 的 xor；p6_op_4 是 402546 的 shl $5、402549 的 xor $0x5b9e 与 40254e 的 add；p6_op_5 先用 402566 的 leal (, %rdi, 8) 与 402570 的 sub %edi 得到七倍，再用 40256d 的 shl $5 加 0xa52b，最后 402577 异或七倍值。说明六个算子都是 32 位的移位、异或、加减组合，只有 op_2 不带常数，op_5 同时用到七倍与三十二倍两种缩放。

八、旋转不是普通移位。 roll 与 rorl 是循环移位，移出的位会补回另一端，4024f6 的左旋七位写成本位移位会丢掉高七位，402526 的右旋一位同理。说明 C 里要写成 (x << 7) | (x >> 25) 与 (x >> 1) | (x << 31)，不能只写一半。

九、位宽与收尾。 参与运算的全是 32 位寄存器，返回值也按 32 位取；4025bf 的 addl $0xcb90,%eax 排在 4025be 的 leave 之后，说明它只改 eax 与栈无关。

Secret 推理流程

这一关和前六关完全不同：它的答案不在命令行参数里，secret_phase 也不是从第一关排到第六关的那种顺序调用，而是 main 在六关全过之后才跳进去的。所以我的第一步是确认它到底从哪里进入。一开始我直接写了 objdump -d --disassemble=secret ./bomb > secret.S，发现文件里只剩几个段头，没有任何指令；后来执行 objdump -t ./bomb | grep -i secret 才看到，真正的符号叫 secret_phase，另外还有 secret_gate、secret_mix、secret_copy。

符号找到后，地址是 0x402630 到 0x4027b9 这四个函数紧密排列的一整段。上界我取 0x4027ba，依据是把符号表里的 size 加了一遍：secret_gate 起于 0x402720、长度 153，结束在 0x4027b9，而 0x4027c0 已经是 challenge_from_seed。这样 --start-address=0x402630 --stop-address=0x4027ba 就能把四个函数完整框进来。

接着是找入口。因为 secret_phase 不接收任何来自命令行的东西，只能反查谁调用它。在 .text 里把所有 0xe8 开头的字节按"目标地址 = 当前地址 + 5 + rel32"算一遍，指向 0x402720 的只有一处 0x401655，在 main 里面。往前读几条，发现：0x401634 先把 %esi 置 7，用 lea -0x8c0(%rbp),%rdi 调 phase_challenge，返回值经 %rax 进 %rsi；0x401648 把 0x402820 的返回值放进 %rdx；lea -0x730(%rbp),%rdi 是在文件里写的第七行。三个参数分别是输入行、challenge、context。

phase_challenge 在 0x402c54 先看 [rdi+0x18]，只有它为 0 时才把 [rdi+8] 当 seed 去调 challenge_from_seed(seed, phase)，而 0x402980 的 movq $1,-0x70(%rbp) 说明默认 seed 是 1，于是 phase 取 7 时 challenge 是 0x85e7bb0f12278575，这个可以离线算出。context 来自 0x402820 的 transcript_context，会遍历六个答案字符串，把里面每一段十进制数字依次送进 splitmix64。说明 context 是前六关答案文本的函数。

secret_gate 的第一道门是长度：0x40275c 调 strlen，接着 sub $0x24,%rax，再用 cmp $0x7f 与 0x402769 的 jbe 做无符号比较。约束为 36 到 163。

第二道门是我一开始读错的地方。lea -0xd0(%rbp),%r12 给出栈缓冲起点，rep stos 把它清零，而 0x40278b 比较的是 -0xb0(%rbp)：cmpl $0x4e45504f,-0xb0(%rbp)，两个地址差了 0x20。我最初以为 OPEN 在开头，后来经测试发现真正的意思是前面 32 字节只是无校验的填充，第 33 字节起才是 OPEN，而 0x4027a4 传给 secret_phase 的 lea -0xac(%rbp),%rdi 正好是 buf+0x24，即 OPEN 之后的那一位。

进了 secret_phase，先是 0x40265f 用 mov $0x3,%esi 调 valid_u32_fields，0x40268a 取 0x403183 处的格式串（内容是 %u %u %u），0x40269b 再核对 sscanf 返回 3。所以口令是三个 uint32。

第一个字段：0x4026a0 到 0x4026a5 是 mov %r13d,%ecx 加 and $0x3,%ecx，然后在 0x4026a8 和 -0x2c(%rbp) 比较。

后两个字段要先进行拆位：0x4026ba 与 0x4026c2 合起来是 (challenge>>32) ^ challenge，0x4026be 与 0x4026c5 合起来是 (context>>32) ^ context，两个都按 32 位取。0x4026c8 再把 %r13 右移 16 位，0x4026ce 用 xor 并进 0xf684*c0 那一项。

字段 1 是三段异或：ror1(h_a^h_b) ^ 0x886e ^ 0xf684*c0 ^ (context>>16)，对应 0x4026d9 的 xor %r9d,%eax，随后在 0x4026dc 和 -0x28(%rbp) 比较。

字段 2。0x4026e1 之后 %eax 没有重新计算，0x4026ed 直接 xor %edx,%eax，用的是字段 1 的结果；同一区间的 %esi 也已经被 0x4026c2 写成了 h_a，0x4026e1 再对它 shr $1。所以第二段公式是在第一段的基础上继续算的，c2 = c1 ^ ((0x7eea + h_b + c0) + (h_a>>1))。

构造输入：32 个非空白字符、OPEN、三个十进制数。seed 42 下得到的是 32 个点接 OPEN3 2087502272 3552943993，整行 59 字节，长度和 OPEN 的位置都落在前面两道门的要求内。

引用资料:
参考资料主要是课程课件，以及 CS:APP 第 3 章「程序的机器级表示」中关于栈帧布局、调用约定和跳转表实现的部分，读反汇编时用它们对照各函数的入口与收尾序列。
使用 WorkBuddy 生成了一份语法速查表：



以及多处测试发现错误时的分析。